

Lab - Azure Blobs - Change Feed

To Begin with the Lab

Summary

This lab demonstrates the **Azure Blob Change Feed** feature. Change Feed records every change made to blobs and blob metadata in an **ordered, durable, immutable, and read-only log**. Applications can read these events to track blob creations, updates, and deletions for auditing, monitoring, and event processing.

Understanding

Azure Blob Change Feed automatically captures all changes that occur in a storage account. Instead of continuously checking blobs for modifications, applications can read the **change feed log** to identify what has changed.

The change feed guarantees that events are stored in the order they occurred. The logs are durable, meaning they remain available even after a system restart, and immutable, meaning they cannot be modified after being written.

To use the feature:

- Enable **Blob Change Feed** in the Storage Account's **Data Protection** settings.
- Azure starts recording blob events.
- A .NET application can read these events using the **Azure.Storage.Blobs.ChangeFeed** package.

Since Change Feed processing is asynchronous, there may be a delay of a few minutes before newly generated events become available.

Example:

Suppose a company stores invoices in Azure Blob Storage.

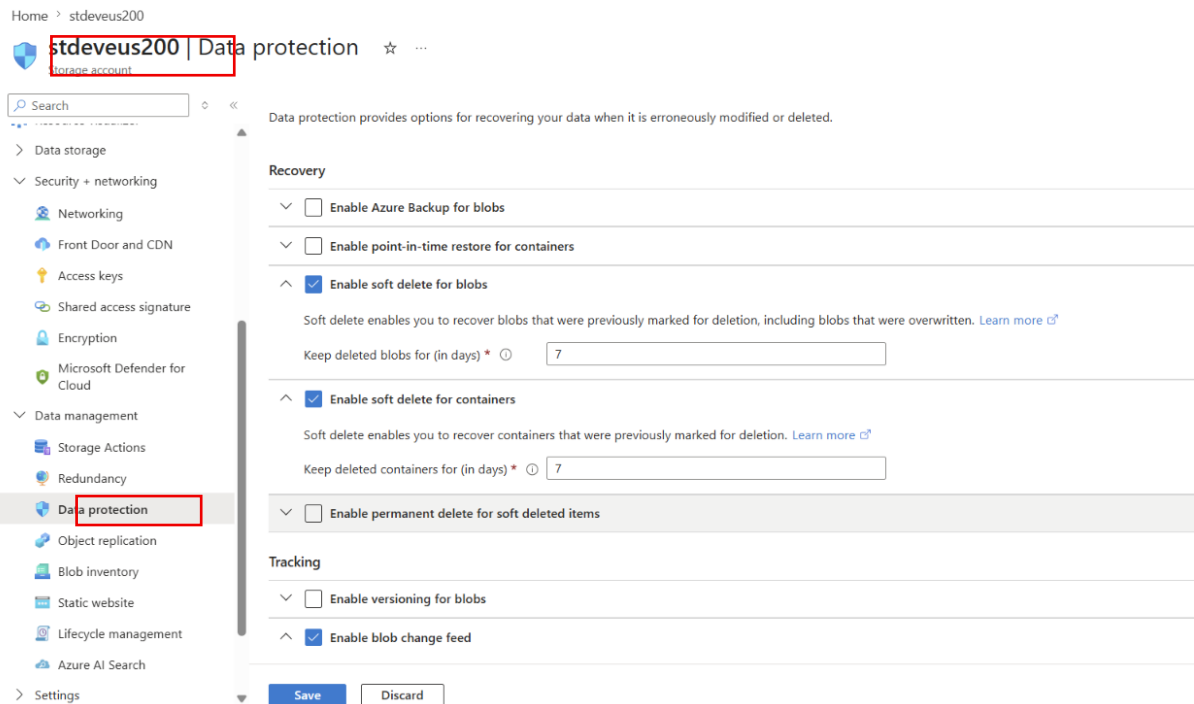
- A user uploads Invoice101.pdf.
- Azure records a **BlobCreated** event in the Change Feed.
- Later, the invoice is updated.
- Azure records a **BlobUpdated** event.
- An auditing application reads the Change Feed and logs every change without directly monitoring the storage account.

This provides a complete history of blob activity.

Lab Steps

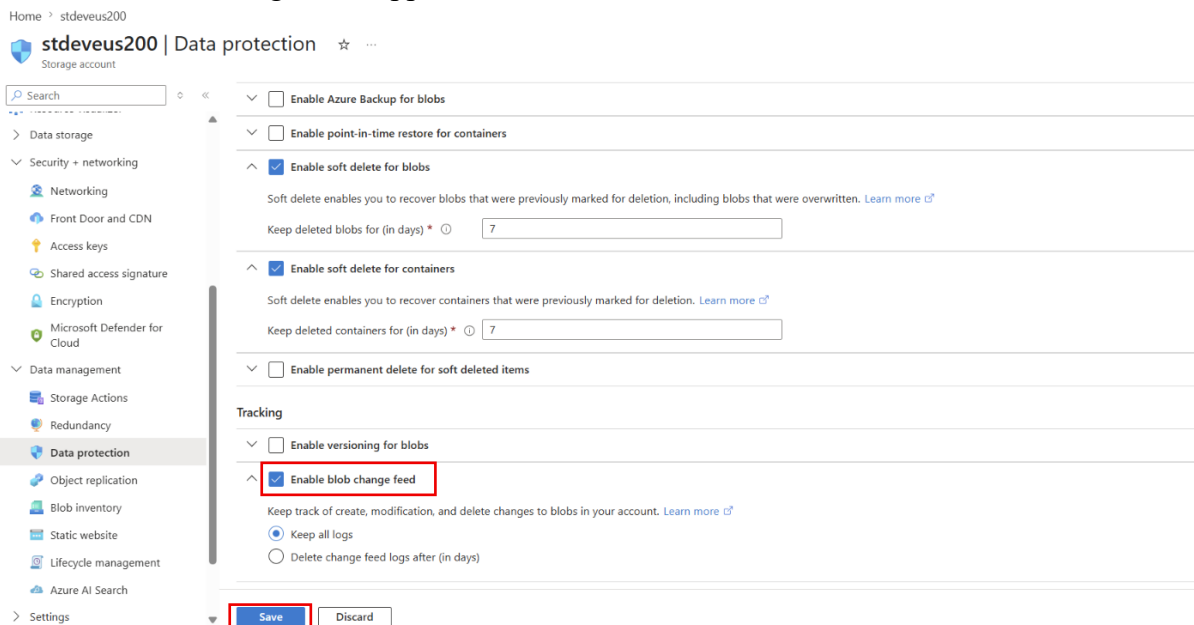
Step 1: Open the Azure Storage Account

- Sign in to the Azure portal.
- Open the desired **Azure Storage Account**.
- Navigate to **Data Management** → **Data Protection**.



Step 2: Enable Blob Change Feed

- Locate the **Blob Change Feed** option.
- Enable the feature.
- Choose whether to keep logs indefinitely or automatically delete them after a specified number of days.
- Click **Save**.
- Wait for the settings to be applied.



Step 3: Create the .NET Console Application

- Create a console application.
- Add the required Azure Storage packages.
- Configure the storage account credentials.

Packages Used

Azure.Storage.Blobs

Azure.Storage.Blobs.ChangeFeed (Pre-release)

```
Program.cs X
change-feed > Program.cs
1 using Azure;
2 using Azure.Storage;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.ChangeFeed;
```

Step 4: Configure Storage Account Access

- Specify the Storage Account Name.
- Specify the Account Key.

```
Program.cs X
change-feed > Program.cs
1 using Azure;
2 using Azure.Storage;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.ChangeFeed;
5
6 string accountName = "stdevs200";
7 string accountKey = "/ShwLG1jCoInKAY0/lzrplwqvxHGv1SRfjhCaDqQKLqoP9j1wMstFYlxsE4U+y2n1/EzT+pBYS8xU+ASTpeMgWQ==";
8
9 var endpoint = new Uri($"https://{accountName}.blob.core.windows.net");
10 var credential = new StorageSharedKeyCredential(accountName, accountKey);
11 var client = new BlobServiceClient(endpoint, credential);
12
13 string? cursor = null;
```

- Create the Storage Account endpoint.
- Create a StorageSharedKeyCredential.
- Create a BlobServiceClient.

```
Program.cs X
change-feed > Program.cs
1 using Azure;
2 using Azure.Storage;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.ChangeFeed;
5
6 string accountName = "stdevs200";
7 string accountKey = "/ShwLG1jCoInKAY0/lzrplwqvxHGv1SRfjhCaDqQKLqoP9j1wMstFYlxsE4U+y2n1/EzT+pBYS8xU+ASTpeMgWQ==";
8
9 var endpoint = new Uri($"https://{accountName}.blob.core.windows.net");
10 var credential = new StorageSharedKeyCredential(accountName, accountKey);
11 var client = new BlobServiceClient(endpoint, credential);
12
13 string? cursor = null;
```

Step 5: Create the Blob Change Feed Client

- Create a BlobChangeFeedClient.
- Connect it to the Blob Service Client.
- Request the available change feed events.

```

12
13 string? cursor = null;
14
15 (cursor, List<BlobChangeFeedEvent> events) =
16     await ChangeFeedResumeWithCursorAsync(client, cursor);
17
18     foreach (var e in events)
19     {
20         Console.WriteLine($"{e.EventType} | {e.Subject} | {e.EventData?.BlobOperationName}");
21     }
22
23
24 static async Task<(string? cursor, List<BlobChangeFeedEvent>>) ChangeFeedResumeWithCursorAsync(
25     BlobServiceClient client,
26     string? cursor)
27 {
28     BlobChangeFeedClient changeFeedClient = client.GetChangeFeedClient();
29     List<BlobChangeFeedEvent> changeFeedEvents = new List<BlobChangeFeedEvent>();
30
31     IAsyncEnumerator<Page<BlobChangeFeedEvent>> enumerator = changeFeedClient
32         .GetChangesAsync(continuationToken: cursor)
33         .AsPages(pageSizeHint: 10)
34         .GetAsyncEnumerator();
35
36     bool hasPage = await enumerator.MoveNextAsync();
37
38     if (!hasPage || enumerator.Current is null)
39     {
40         return (cursor, changeFeedEvents);
41     }

```

Step 6: Read the Change Feed

- Retrieve the list of change events.
- Iterate through each event.
- Display information such as:
 - Event Type
 - Subject
 - Blob Name
 - Event Time
 - Metadata (if available)

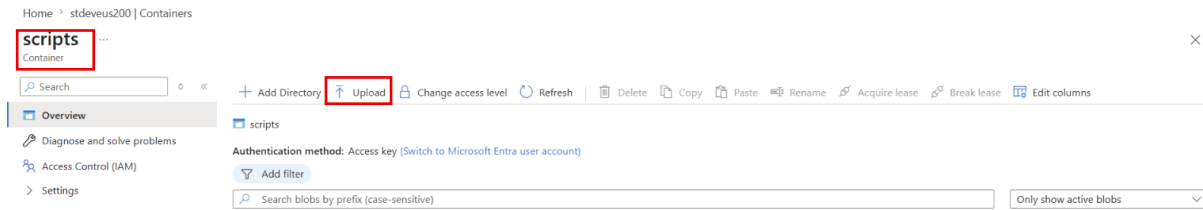
```

12
13 string? cursor = null;
14
15 (cursor, List<BlobChangeFeedEvent> events) =
16     await ChangeFeedResumeWithCursorAsync(client, cursor);
17
18     foreach (var e in events)
19     {
20         Console.WriteLine($"{e.EventType} | {e.Subject} | {e.EventData?.BlobOperationName}");
21     }
22
23
24 static async Task<(string? cursor, List<BlobChangeFeedEvent>>) ChangeFeedResumeWithCursorAsync(
25     BlobServiceClient client,
26     string? cursor)
27 {
28     BlobChangeFeedClient changeFeedClient = client.GetChangeFeedClient();
29     List<BlobChangeFeedEvent> changeFeedEvents = new List<BlobChangeFeedEvent>();
30
31     IAsyncEnumerator<Page<BlobChangeFeedEvent>> enumerator = changeFeedClient
32         .GetChangesAsync(continuationToken: cursor)
33         .AsPages(pageSizeHint: 10)
34         .GetAsyncEnumerator();
35
36     bool hasPage = await enumerator.MoveNextAsync();
37
38     if (!hasPage || enumerator.Current is null)
39     {
40         return (cursor, changeFeedEvents);
41     }

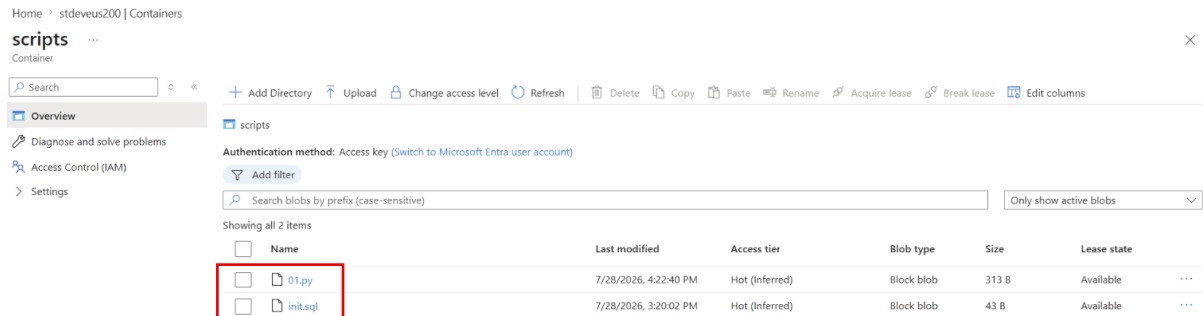
```

Step 7: Generate a Blob Change

- Open a Blob Container.
- Click on the Upload.

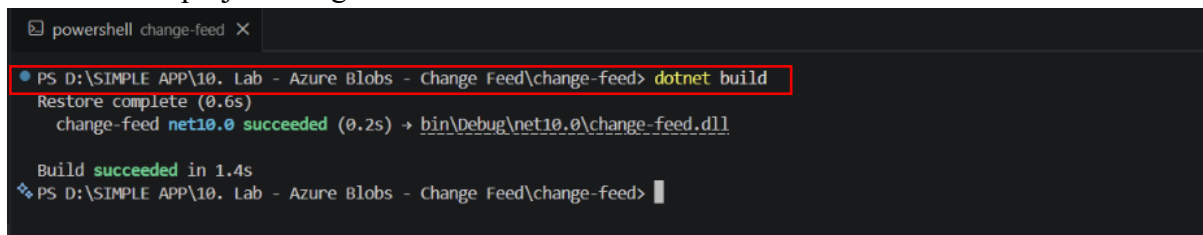


- Upload a file from the local machine.
- Wait a few minutes for Azure to record the change.



Step 8: Build and Run the Application

- Build the project using .NET build.



- Run the application using .NET run.
- Blob uploaded Successfully.

